

《人工智能与深度学习》课程

实验报告

学号：04022212

姓名：钟 源

2022 年 05 月 06 日

实验四：循环神经网络

一、实验目的

搭建循环神经网络 RNN(Recurrent Neural Network)来实现语音降噪，将搭建一个简易的基于门控循环单元 GRU(Gated Recurrent Unit)的 RNN 网络对被 babble、car、train、restaurant、street 五种噪声污染的语音信号进行降噪处理。

二、实验内容

完成对含噪语音数据的降噪处理。注意给出实验代码和详细过程介绍。

1. 训练前准备

1) 导入第三方库：

```
import torch
from torch.utils.data import Dataset
import torchaudio
from IPython.display import Audio, display
import librosa
from torch import nn
from torch.utils.data import DataLoader
import os
import matplotlib.pyplot as plt
import numpy as np
import glob
import torch.nn.functional as F
from numpy import random
```

2) 读取 NOIZEUS 数据集:

Noizeus 是 MIT 的 Loizou 实验室录制的语音数据库,旨在用来对比不同的语音增强算法性能。数据集已在工作目录./data/noizeus 下,本次实验使用数据集中的./clean 和./withnoise 两个目录下的语音文件,通过以下方式进行数据集的加载。

```
#读取训练文件
filelist=[] # to store noisy audio
labellist=[] # to store clean audio

noisypath="./data/noizeus/withnoise/"
cleanpath='./data/noizeus/clean/'

noisyfolders=os.listdir(noisypath)
cleanfolders=os.listdir(cleanpath)

for folder in noisyfolders:
    audios=os.listdir(noisypath+folder+"/")
    for audio in audios:
        filelist.append(noisypath+folder+"/"+audio)

for audio in cleanfolders:
    labellist.append(cleanpath+audio)

print('read {} noisy files.'.format(len(filelist)))
print('read {} clean files.'.format(len(labellist)))
```

结果如下:

```
read 450 noisy files.
read 30 clean files.
```

3) 数据预处理:

对于语音信号,往往不直接将时域波形当作特征 Feature 送入神经网络、而是应提取对应的时频表示。最常用的时频表示称为 STFT,由于人耳对声音的感知主要由短时幅度谱决定,故将 STFT 的幅度谱这一重要特征称为语谱图(spectrogram)。本实验取每句话的前 16704 个样本点,转换为一张(129,258)的语谱图,再将其分割为两张(129,129)的子语谱图。

```
def spectrogram(wav):
    win_length=256
    n_fft=256
    hop_length=64
    wav = wav.squeeze()
    S = torch.stft(wav, n_fft=n_fft, hop_length=hop_length,
                   win_length=win_length,
    window=torch.hann_window(window_length=win_length),
```

```

        center=True, return_complex=True)
magnitude = torch.abs(S)          # 振幅
phase = torch.exp(1j * torch.angle(S)) # 相位
return magnitude, phase

```

4) 准备数据集 Dataset:

在 pytorch 中,对于原始数据信息,需要自定义一个 Dataset 类,来定义 pytorch 在训练时如何读取数据。自定义的 Dataset 类需要继承 Dataset 基类,并复写其中的 `__init__()`,`__getitem__()`,`__len__()`三个特殊方法。

在本实验中,第 `index` 个数据应当为一个由输入特征(一张(129,129)的语谱图)和目标特征(一个(129,129)的矩阵)组成的数据对,这里的[0]、[1]分别是数据对中的含噪谱图和真实谱图。

```

#构建 pytorch 数据集
class mydataset(Dataset):
    def __init__(self,filelist,labellist):
        self.filelist=filelist
        self.labellist=labellist
        self.win_length=256
        self.hop_length=64
        self.n_fft=256

    def __len__(self):
        return len(self.filelist*2)

    def __getitem__(self, idx):
        f=idx//2
        n=idx%2

        noised_waveform,_ = torchaudio.load(self.filelist[f])
        noised_waveform = noised_waveform[0][:16704]#[16704,]
        noised_waveform=noised_waveform.unsqueeze(0)#[1,16704]

        clean_waveform,_=torchaudio.load(self.labellist[int(filelist[f][-6:-4])-1])
        clean_waveform = clean_waveform[0][:16704]#[16704,]
        clean_waveform=clean_waveform.unsqueeze(0)#[1,16704]

        noise_waveform=noised_waveform-clean_waveform

        noised_mag ,_ = spectrogram(noised_waveform)
        clean_mag ,_ = spectrogram(clean_waveform)
        noise_mag,_ = spectrogram(noise_waveform)

        #语谱图 shape[129, 258],截成两张[129,129]

```

```

    postion=n
    length=129
    sliced_noised=noised_mag[:,length*postion:length*(postion+1)]
    sliced_clean=clean_mag[:,length*(postion):length*(postion+1)]
    sliced_noise=noise_mag[:,length*(postion):length*(postion+1)]
    #single_frame_noised_mag= sliced_noised[:,length:2*length]

    _eps = torch.finfo(torch.float).eps # 防止分母出现 0
    mask_IRM = torch.sqrt(sliced_clean ** 2/(sliced_noise ** 2+sliced_clean **
2+_eps)) # IRM, 模型输出
    label=mask_IRM #掩蔽

    return sliced_noised.unsqueeze(0),label.unsqueeze(0)

#实例化 MyDataset 类
dataset=mydataset(filelist,labellist)
#查看 dataset 值
print(dataset[2][0].shape)
print(dataset[2][1].shape)

```

结果如下：

```

torch.Size([1, 129, 129])
torch.Size([1, 129, 129])

```

5) 准备数据分发 DataLoader :

理论上，只要给出要读取的数据的索引 index，就能从 MyDataset 中读出第 index 个数据对。但是，实际在网络训练中，往往不是一个数据对、一个数据对地送入网络，而是将数据一批一批地送入网络。每批的个数就是 batchsize。并且，往往我们还需要对数据打乱处理以避免过拟合。在 pytorch 框架中，内置了负责一批一批地分发数据的 DataLoader 类。

首先，我们先分割训练数据集中的一部分作为验证集。然后再定义 Dataloader 类，它接收上文定义好的数据集 dataset，按照 batchsize 大小，一批一批地从 dataset 中取出数据、打包分发。

```

#按照 0.8 0.2 比例分割训练集和验证集
length=len(dataset)
train_size = int(0.8 * len(dataset))
validate_size = len(dataset) - train_size
train_set,validate_set=torch.utils.data.random_split(dataset,[train_size,validate_s
ize])
batch_size=16
train_dataloader = DataLoader(train_set, batch_size=batch_size,shuffle=True)
validate_dataloader = DataLoader(validate_set, batch_size=batch_size,shuffle=True)

```

2. 训练 RNN

1) 构建网络:

在本实验中，使用一个 8 层的 RNN 网络来实现语音降噪任务。其输入为一张 129*129 的语谱图，即二维矩阵；其输出也为 129*129 的矩阵。注意到矩阵的两个维度是具有物理含义的，即**时间轴**和**频率轴**。因此，可以每次将语谱图中的一帧，即 129*1 的一个向量送入网络，经网络得到 129*1 的输出。再将同样的操作重复 128 次，共 129 个时间步。注意，RNN 与普通的全连接层的区别，正是体现在：**网络每一层的参数也随着时间步不断更新**。

```
#8 层 GRU 输入格式要求[batch channel sequence feature]
class RNN(nn.Module):
    def __init__(self):
        super(RNN, self).__init__()

        self.gru1=nn.GRU(129,hidden_size=1024,num_layers=1,batch_first=True) # 第1层
        self.gru2=nn.GRU(1024,hidden_size=1024,num_layers=6,batch_first=True) # 第2-7 层
        self.gru3=nn.GRU(1024,hidden_size=129,num_layers=1,batch_first=True) # 第8层
        self.sig=nn.Sigmoid() # 通过 Sigmoid 激活函数 输出（值域 [0,1]，适合概率或掩码预测）

    def forward(self, x):
        n_channels, n_f_bins, n_frame_size = x.shape[-3],x.shape[-2],x.shape[-1]
        gru_in = x.reshape(-1, n_channels * n_f_bins, n_frame_size).transpose(-1,-2)
        gru_out,_ = self.gru1(gru_in) # (B, S, C*F) → (B, S, 1024)
        gru_out,_ = self.gru2(gru_out) # (B, S, 1024) → (B, S, 1024)
        gru_out,_ = self.gru3(gru_out) # (B, S, 1024) → (B, S, 129)
        gru_out = gru_out.transpose(-1,-2).reshape(-1, n_channels, n_f_bins, n_frame_size)
        out=self.sig(gru_out)
        return out

model = RNN().to(device)
```

这样，就可以用 `model(input)`，直接获取网络的输出结果，框架会自动调用 `model` 的 `forward` 方法来前向传播。可以通过 `print` 查看网络及其输出结构：

```
#查看输出 shape
x=torch.rand([16,1,129,129]).to(device)
print(model)
print(model(x).shape)
```

结果如下：

```
RNN(
  (gru1): GRU(129, 1024, batch_first=True)
  (gru2): GRU(1024, 1024, num_layers=6, batch_first=True)
  (gru3): GRU(1024, 129, batch_first=True)
  (sig): Sigmoid()
)
torch.Size([16, 1, 129, 129])
```

2) 定义 loss, optimizer, train 和 test 函数:

先定义在训练中要使用的 loss 函数（均方误差损失）和 optimizer（Adam，学习率为 $1e-4$ ），如下。

```
loss_fn = nn.MSELoss(reduction='mean')
optimizer = torch.optim.Adam(model.parameters(), lr=1e-4,)
```

接下来定义 train 函数，在这个函数中完整执行训练过程。

```
def train(dataloader, model, loss_fn, optimizer):
    size = len(dataloader.dataset)
    model.train()
    for batch, (X, y) in enumerate(dataloader):
        X, y = X.to(device), y.to(device)

        # 计算损失函数
        pred = model(X)
        loss = loss_fn(pred, y)

        # 反向传播
        optimizer.zero_grad()
        loss.backward()
        optimizer.step()

        if batch % 10 == 0:
            loss, current = loss.item(), batch * len(X)
            print(f"loss: {loss:>7f}  [{current:>5d}/{size:>5d}]")
```

类似地，定义用于测试时使用的 test 函数。与 train 函数基本一致，区别在于 test 阶段不需要反向传播。

```
def test(dataloader, model, loss_fn):
    size = len(dataloader.dataset)
    num_batches = len(dataloader)
    model.eval()
    test_loss, correct = 0, 0
    with torch.no_grad():
        for X, y in dataloader:
            X, y = X.to(device), y.to(device)
            pred = model(X)
            test_loss += loss_fn(pred, y).item()
    test_loss /= num_batches
    print(f"Test Error: \n Avg loss: {test_loss:>8f} \n")
```

3) 训练模型:

附件./model 目录下已经事先保存了一个已训练 15 轮的模型 rnn_gru.pt, 实验选择在此模型的基础上继续训练，即首先加载已有的模型参数，再进行 epoch 训

练。（或者也可以重新开始训练，两种训练方式的区别在于加载已有模型的指令是否注释）

```
# 如果需要加载已有模型参数，取消注释
model.load_state_dict(torch.load('./model/rnn_gru.pt',map_location=device))

epochs = 5
for t in range(epochs):
    print(f"Epoch {t+1}\n-----")
    train(train_dataloader, model, loss_fn, optimizer)
    test(validate_dataloader, model, loss_fn)
print("Done!")
```

结果如下：

```
-----
loss: 0.077394 [ 0/ 720]
loss: 0.076750 [ 160/ 720]
loss: 0.075338 [ 320/ 720]
loss: 0.070951 [ 480/ 720]
loss: 0.074768 [ 640/ 720]
Test Error:
Avg loss: 0.073641

Epoch 4
-----
loss: 0.072243 [ 0/ 720]
loss: 0.071931 [ 160/ 720]
loss: 0.073511 [ 320/ 720]
loss: 0.075817 [ 480/ 720]
loss: 0.070995 [ 640/ 720]
Test Error:
Avg loss: 0.072395

Epoch 5
-----
loss: 0.071625 [ 0/ 720]
loss: 0.074532 [ 160/ 720]
loss: 0.071223 [ 320/ 720]
loss: 0.071780 [ 480/ 720]
loss: 0.074790 [ 640/ 720]
Test Error:
Avg loss: 0.071457

Done!
```

4) 保存模型：

训练结束后，保存模型参数。这里使用的是只保存模型参数，不保存模型整体的方法，以节省空间。

```
# Save the model
if not os.path.exists('./model'):
    os.makedirs('./model')
torch.save(model.state_dict(), './model/rnn_gru.pt')
```

对应的加载模型的代码如下。

```
# 测试环节
model = RNN().to(device)
model.load_state_dict(torch.load('./model/rnn_gru.pt',map_location=device))
```

5) 数据可视化函数:

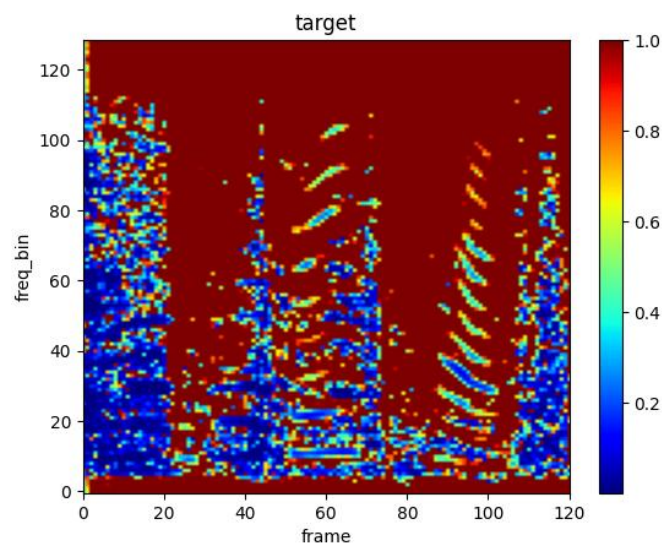
为了直观地验证降噪前后的效果, 需要展示出训练前后的波形、语谱图, 并可以播放试听。绘制语谱图和播放试听的步骤封装在函数中, 方便调用。

```
# 绘制语谱图
def plot_spectrogram(spec, title=None, ylabel='freq_bin', aspect='auto', xmax=None,
                      if_log=True):
    fig, axs = plt.subplots(1, 1)
    axs.set_title(title or 'Spectrogram (db)')
    axs.set_ylabel(ylabel)
    axs.set_xlabel('frame')
    if if_log:
        spec = librosa.power_to_db(spec, ref=np.max)
    im = axs.imshow(spec, origin='lower', aspect=aspect, cmap='jet')
    if xmax:
        axs.set_xlim((0, xmax))
    fig.colorbar(im, ax=axs)
    plt.show(block=False)

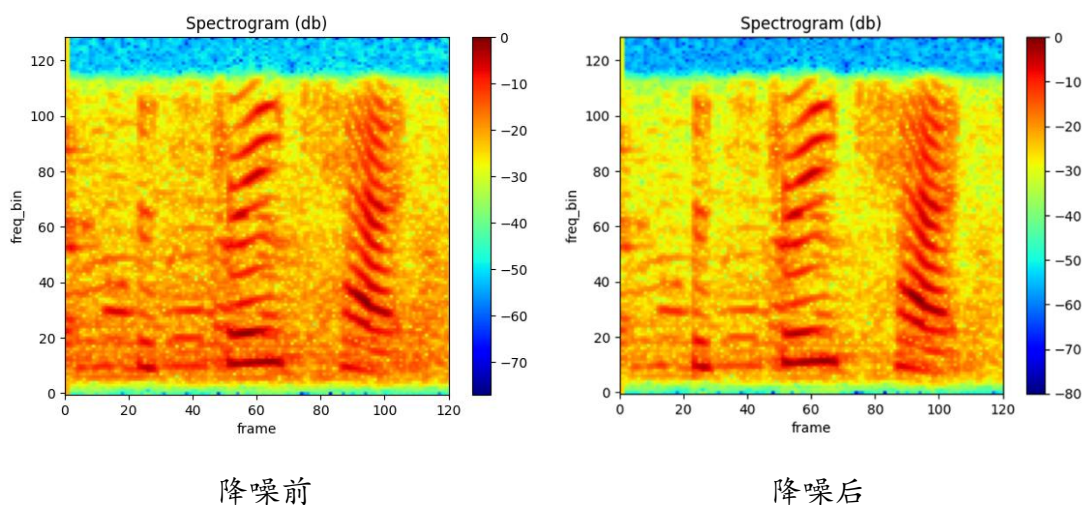
#播放试听
def play_audio(waveform, sample_rate):
    waveform = waveform.numpy()

    num_channels, num_frames = waveform.shape
    if num_channels == 1:
        display(Audio(waveform[0], rate=sample_rate))
    elif num_channels == 2:
        display(Audio((waveform[0], waveform[1]), rate=sample_rate))
    else:
        raise ValueError("Waveform with more than 2 channels are not supported.")
```

绘制的理想纯净语音的语谱图如下:



绘制的降噪前后的语谱图如下：



6) 在新数据上生成预测结果：

加载保存的模型并输入测试数据得到降噪后的语音信号。

```
#从测试音频中挑选一个进行降噪
num=1 #可以为 1, 2, 3
noised_waveform,_= torchaudio.load('./data/mynoise{}.wav'.format(num))
noised_waveform=noised_waveform[:, :16704]
mag,phase=spectrogram(noised_waveform)
phase = phase.to(device)
mag1=mag[:, :129].to(device)
denoised_mag1=mag1*model(mag1.unsqueeze(0).unsqueeze(0))[0]
mag2=mag[:, 129:].to(device)
denoised_mag2=mag2*model(mag2.unsqueeze(0).unsqueeze(0))[0]
denoised_mag=torch.cat([denoised_mag1,denoised_mag2],dim=2)
complex_stft = (denoised_mag * phase).detach().cpu()
waveform=torch.istft(complex_stft, n_fft=256, hop_length=64, win_length=256,
                      window=torch.hann_window(window_length=256),center=True)
```

绘制降噪语音、含噪语音的时域波形、语谱图、播放语音的代码如下：

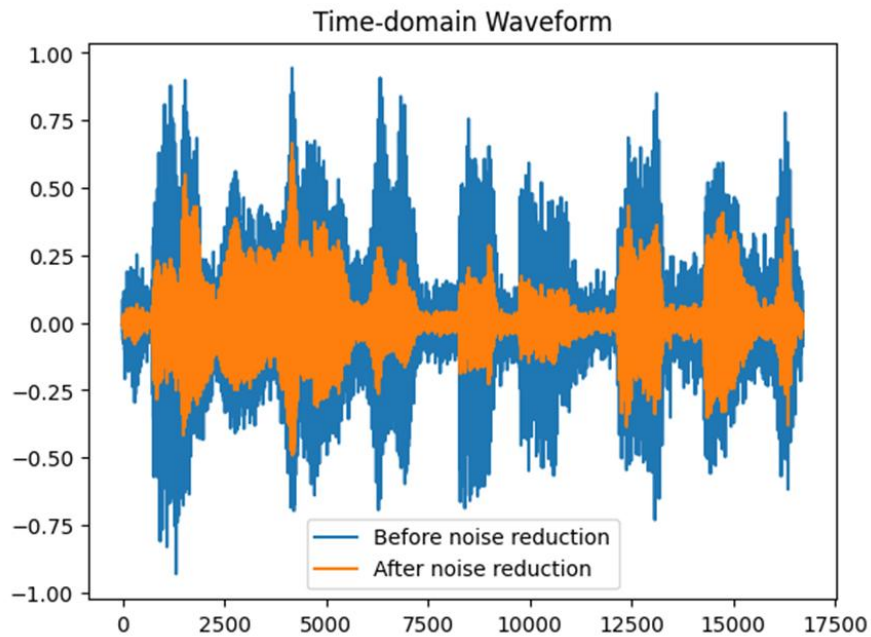
```
# 绘制时域波形
plt.plot(noised_waveform.squeeze().numpy(),label="Before noise reduction")#降噪前波形
plt.plot(waveform.squeeze().numpy(),label="After noise reduction")#降噪后波形
plt.legend()
plt.title("Time-domain Waveform")

# 绘制原始含噪语音的语谱图
noised_mag,_=spectrogram(noised_waveform)
plot_spectrogram(noised_mag,xmax=120)
# 绘制降噪后语音的语谱图
```

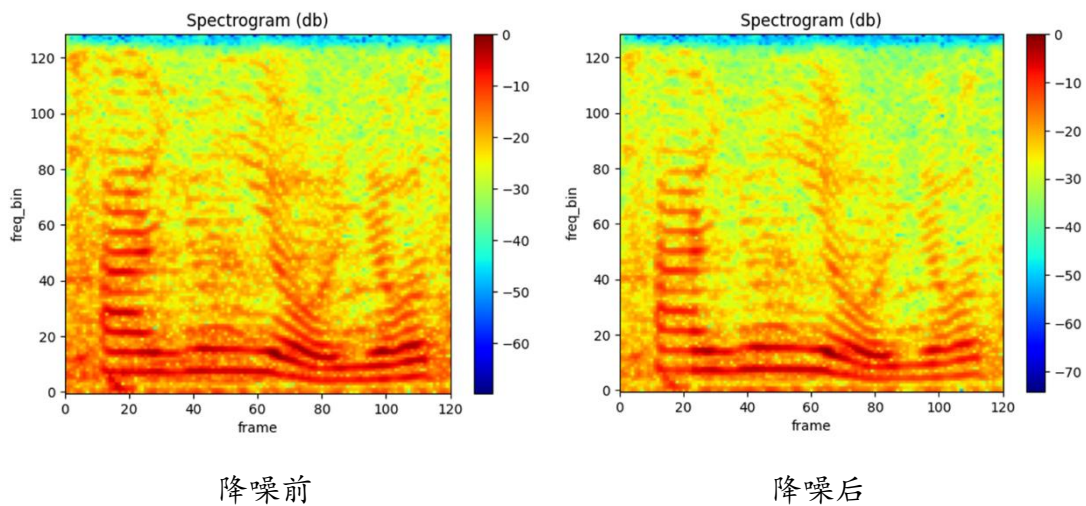
```
mag,_=spectrogram(waveform)
plot_spectrogram(mag,xmax=120)

play_audio(noised_waveform,8000) # 降噪前
play_audio(waveform,8000) # 降噪后
```

绘制的时域波形如下：



绘制的降噪前后的语谱图如下：



三、提高训练

基于实验手册的内容，尝试对现有实验做出自己的尝试和修改，比较不同参数带来的性能影响。

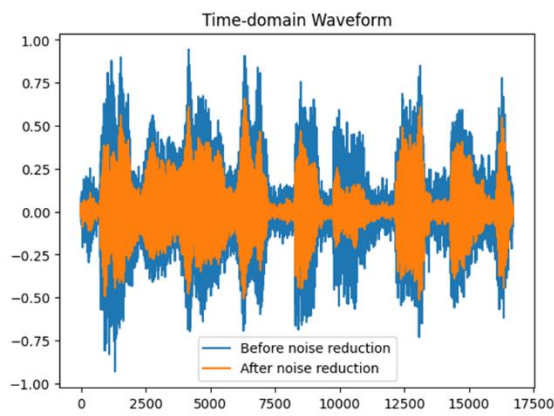
1. 增加训练轮数

首先尝试改变模型的迭代次数，分别设置 epoch 为 3、6、12、24，并记录每次训练的结果。

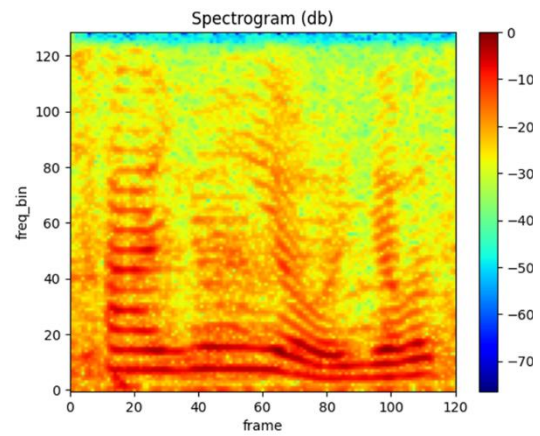
不同的迭代次数最后的损失函数值如下：

迭代次数	损失函数最终值
0	0.142855
3	0.089616
6	0.083248
12	0.074856
24	0.065526

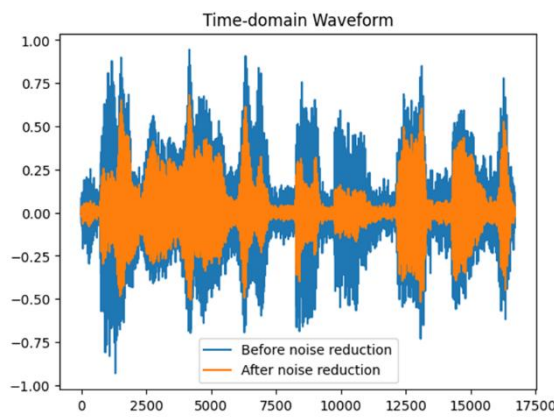
记录降噪后的时域波形和语谱图如下：



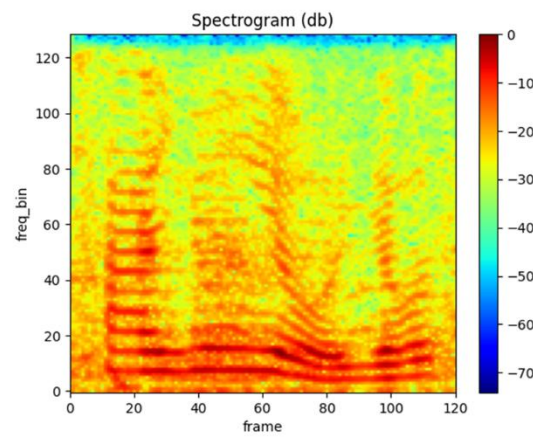
Epoch=3 时的降噪后时域波形



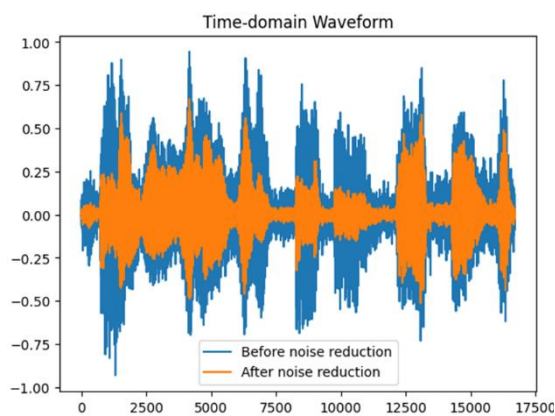
Epoch=3 时的降噪后语谱图



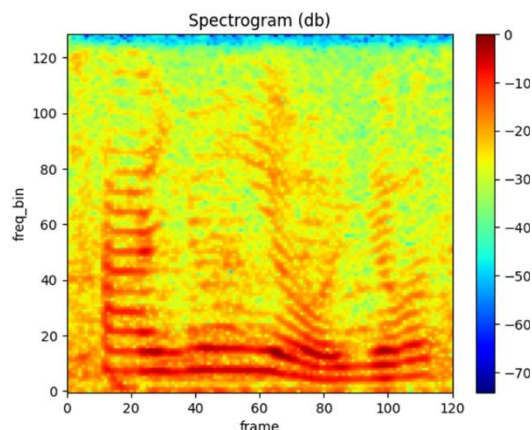
Epoch=6 时的降噪后时域波形



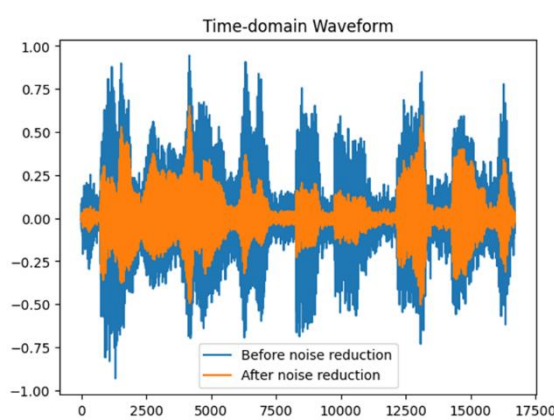
Epoch=6 时的降噪后语谱图



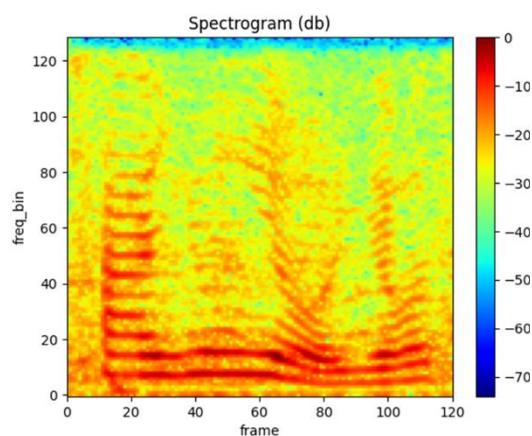
Epoch=12 时的降噪后时域波形



Epoch=12 时的降噪后语谱图



Epoch=24 时的降噪后时域波形



Epoch=24 时的降噪后语谱图

可以看到，随着训练轮数逐渐增加，损失函数最终值逐渐下降，且时域波形中看到毛刺逐渐变小，说明增加训练轮数确实能有效达到更好的降噪效果。不过也注意到，随着训练轮数不断增大，增加的降噪效果慢慢减小，趋于稳定。

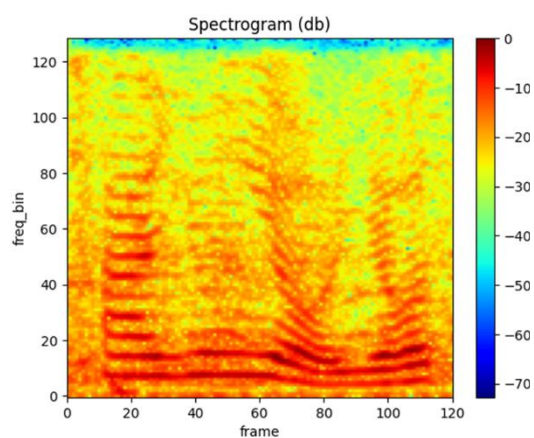
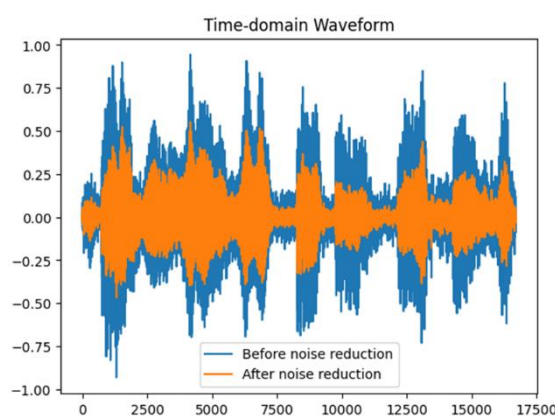
2. 调整学习率

接着重新调整训练轮数为 5，然后设置不同的学习率（分别设置为 $1e-5$ 、 $3e-5$ 、 $1e-4$ 、 $3e-4$ 、 $1e-3$ ），并记录每次训练的结果。

不同的迭代次数最后的损失函数值如下：

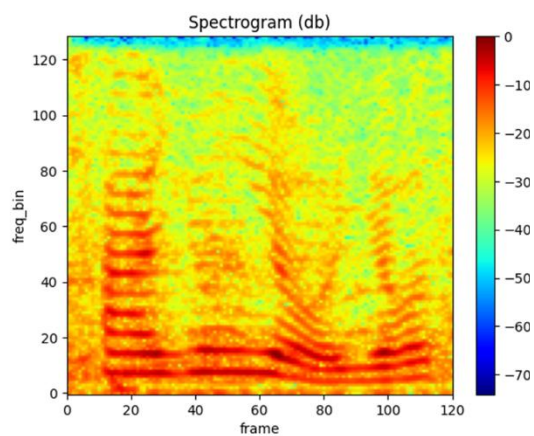
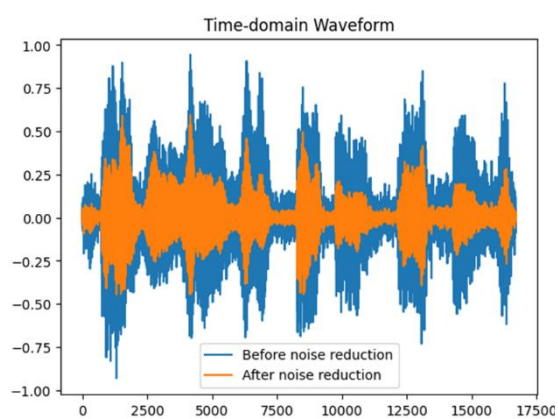
学习率	损失函数最终值
$1e-5$	0.113511
$3e-5$	0.092835
$1e-4$	0.084459
$3e-4$	0.078587
$1e-3$	0.116133

记录降噪后的时域波形和语谱图如下：



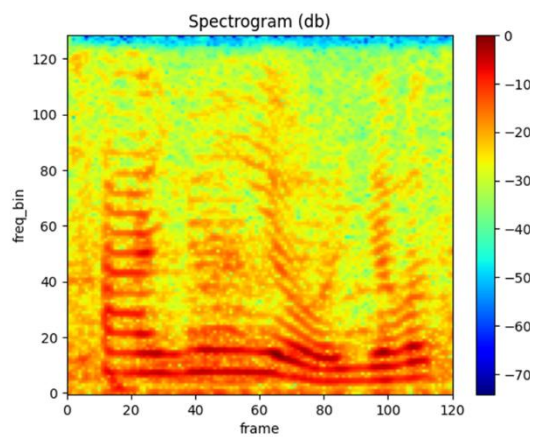
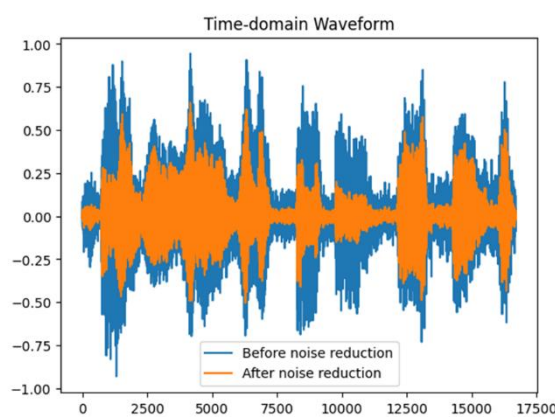
学习率= $1e-5$ 时的降噪后时域波形

学习率= $1e-5$ 时的降噪后语谱图



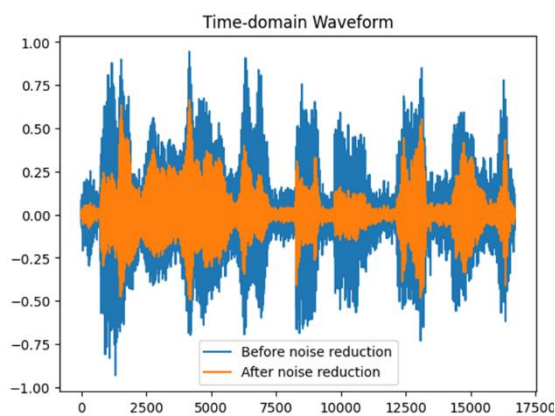
学习率= $3e-5$ 时的降噪后时域波形

学习率= $3e-5$ 时的降噪后语谱图

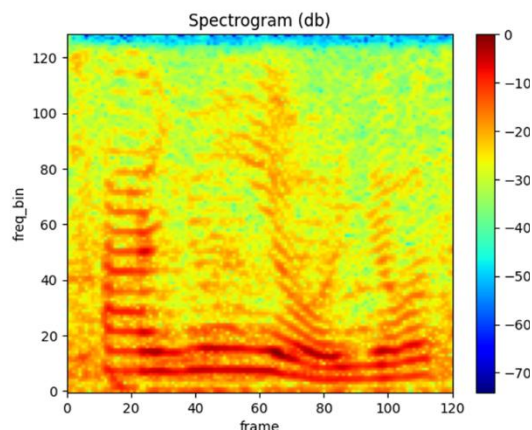


学习率= $1e-4$ 时的降噪后时域波形

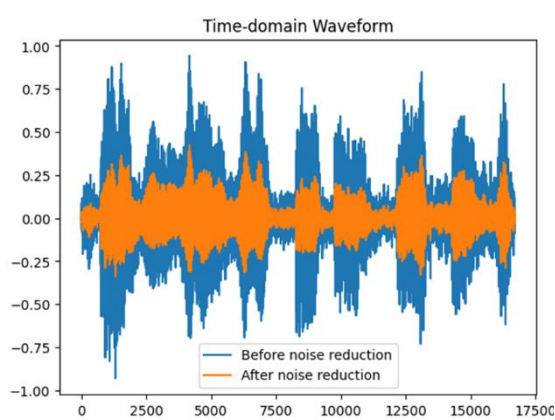
学习率= $1e-4$ 时的降噪后语谱图



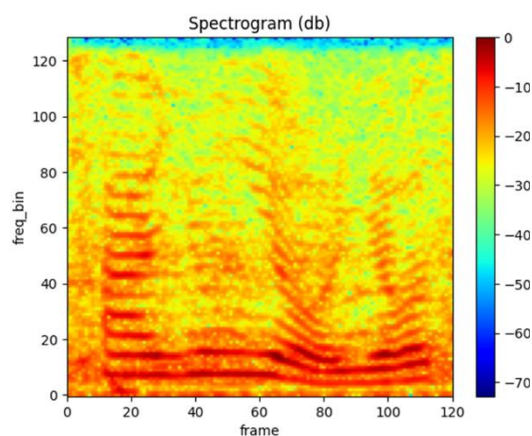
学习率= $3e-4$ 时的降噪后时域波形



学习率= $3e-4$ 时的降噪后语谱图



学习率= $1e-3$ 时的降噪后时域波形



学习率= $1e-3$ 时的降噪后语谱图

可以看到，当训练轮数较小时，更大的学习率可以更快地迭代参数，以得到更好的训练结果。但是当学习率过大时（如 $1e-3$ ），则学习效果不好，无法有效训练模型。这与我们印象相符：较小的学习率会得到更好的精度，但是花费的迭代时间会变长。

3. 改变优化器

接下来我们尝试改变优化器为 SGD，设置学习率为 $1e-5$ ，动量参数为 0.9，训练轮数为 5，迭代结果如下：

```

-----
loss: 0.136688 [   0/ 720]
loss: 0.134293 [  160/ 720]
loss: 0.139352 [  320/ 720]
loss: 0.144784 [  480/ 720]
loss: 0.137825 [  640/ 720]
Test Error:
  Avg loss: 0.140086

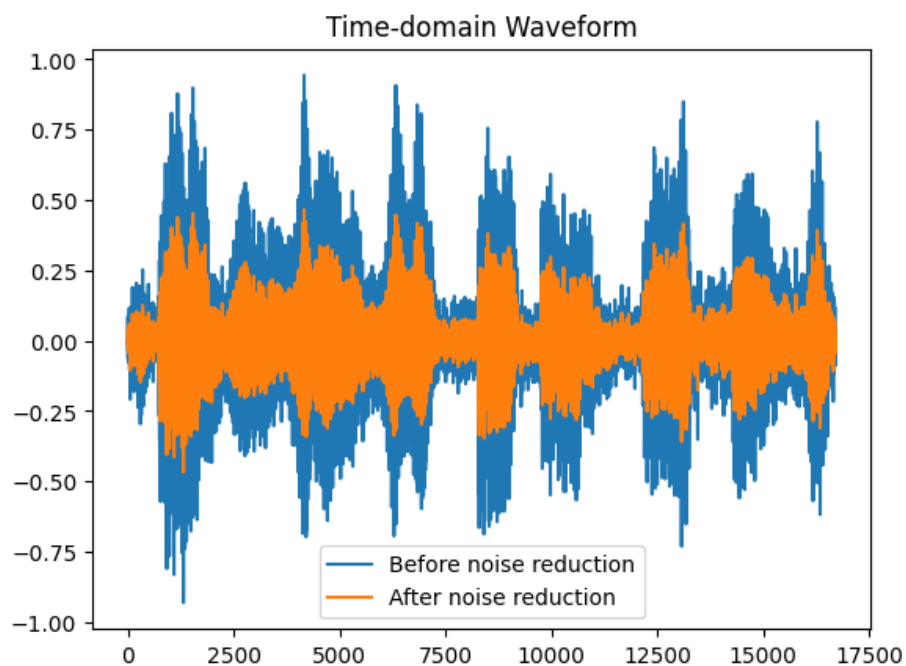
Epoch 4
-----
loss: 0.137901 [   0/ 720]
loss: 0.137423 [  160/ 720]
loss: 0.141318 [  320/ 720]
loss: 0.143418 [  480/ 720]
loss: 0.137991 [  640/ 720]
Test Error:
  Avg loss: 0.140360

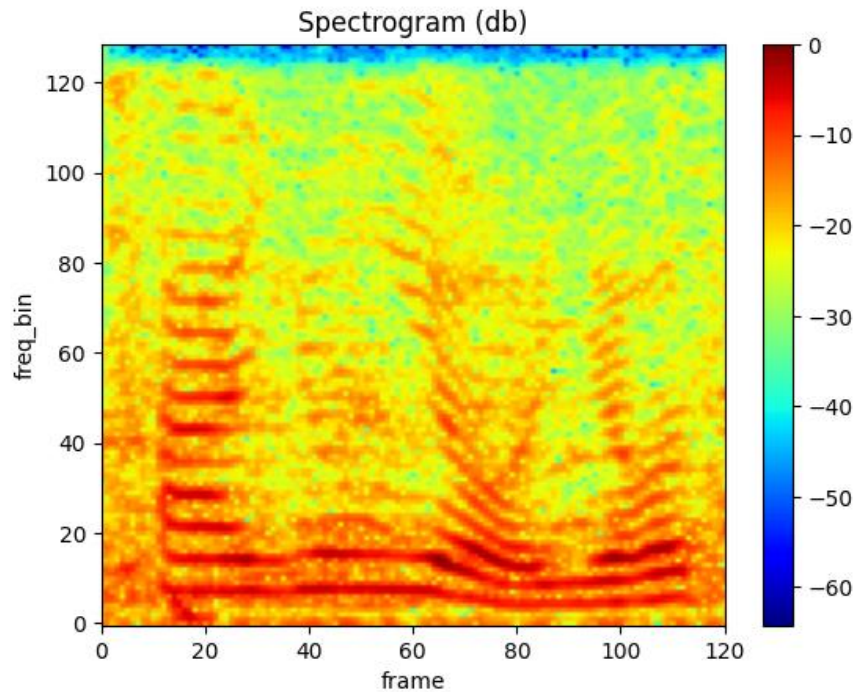
Epoch 5
-----
loss: 0.140614 [   0/ 720]
loss: 0.137520 [  160/ 720]
loss: 0.140259 [  320/ 720]
loss: 0.140632 [  480/ 720]
loss: 0.145828 [  640/ 720]
Test Error:
  Avg loss: 0.139951

Done!

```

绘制的降噪后的时域波形和语谱图如下：





可以看到，哪怕时=是用了较小的学习率，当损失函数值降低到 0.14 左右也无法继续下降了，说明相比于 Adam 优化器，SGD 优化器效果更差。

4. 改变激活函数

接下来我们尝试改变激活函数为 ReLU，设置学习率为 $1e-4$ ，训练轮数为 5，迭代结果如下：

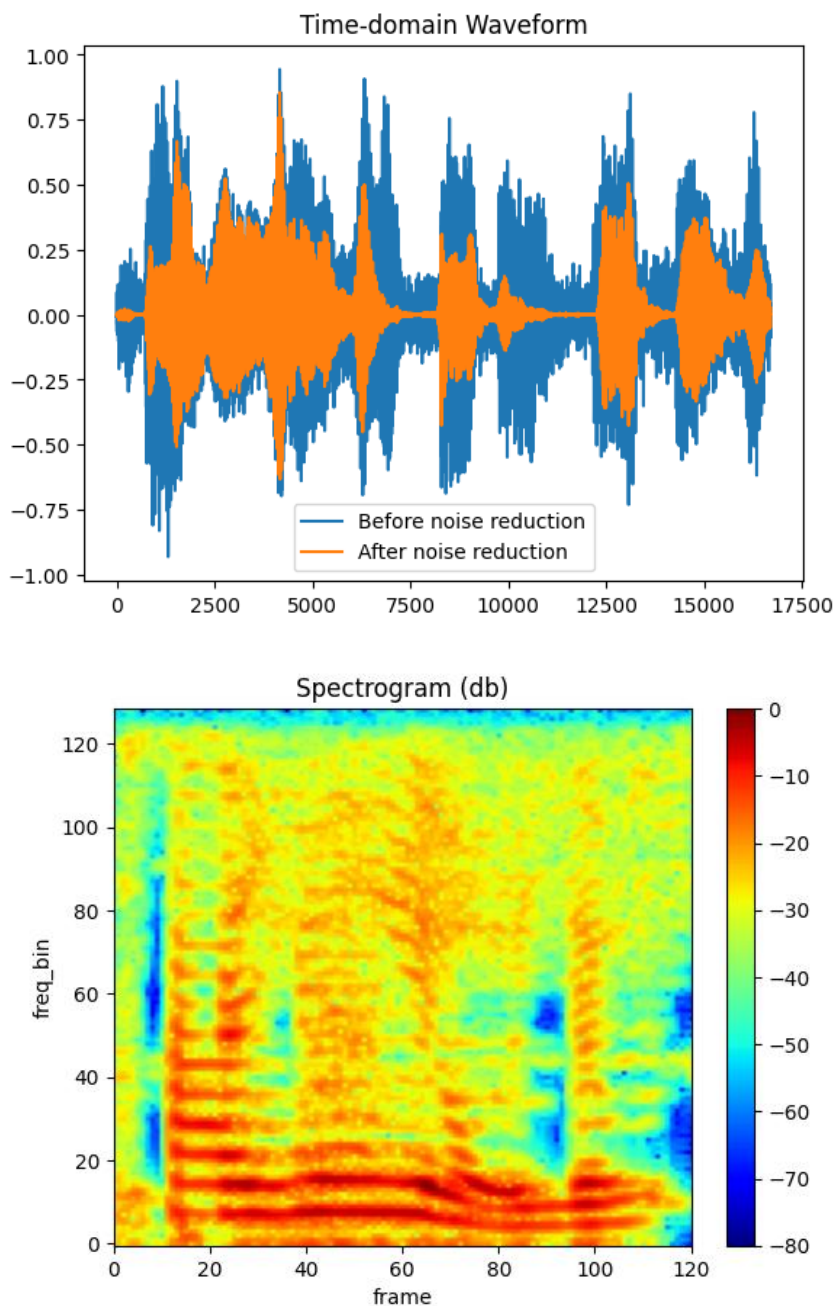
```
-----
loss: 0.109503 [ 0/ 720]
loss: 0.106495 [ 160/ 720]
loss: 0.100537 [ 320/ 720]
loss: 0.097853 [ 480/ 720]
loss: 0.100075 [ 640/ 720]
Test Error:
Avg loss: 0.100625

Epoch 4
-----
loss: 0.095349 [ 0/ 720]
loss: 0.098437 [ 160/ 720]
loss: 0.099678 [ 320/ 720]
loss: 0.088915 [ 480/ 720]
loss: 0.082910 [ 640/ 720]
Test Error:
Avg loss: 0.090627

Epoch 5
-----
loss: 0.091815 [ 0/ 720]
loss: 0.092142 [ 160/ 720]
loss: 0.092849 [ 320/ 720]
loss: 0.082772 [ 480/ 720]
loss: 0.085842 [ 640/ 720]
Test Error:
Avg loss: 0.085729

Done!
```


绘制的降噪后的时域波形和语谱图如下：



可以看到激活函数为 ReLU 时，模型的训练性能与激活函数为 Sigmoid 时大体相当，可见两者同样适合 GRU 网络。

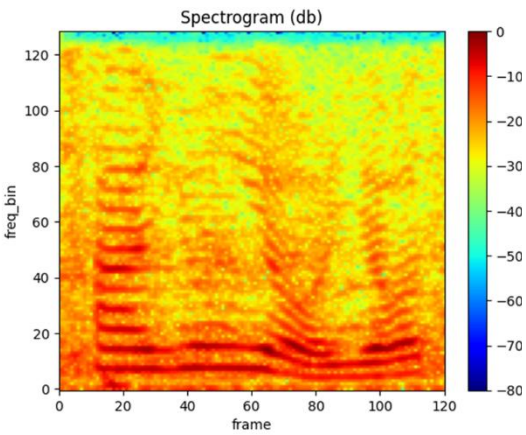
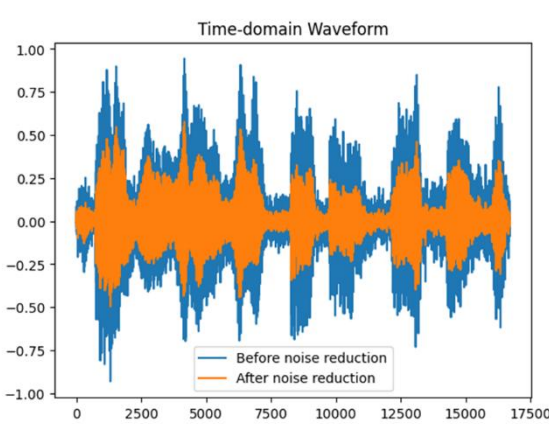
5. 改变网络层数

接下来我们尝试改变 GRU 网络的层数：设置学习率为 $1e-4$ ，训练轮数为 5，激活函数为 sigmoid，分别设置网络总层数为 1，4，8，12，16，然后并记录每次训练的结果。

不同的网络层数下，最后的损失函数值如下：

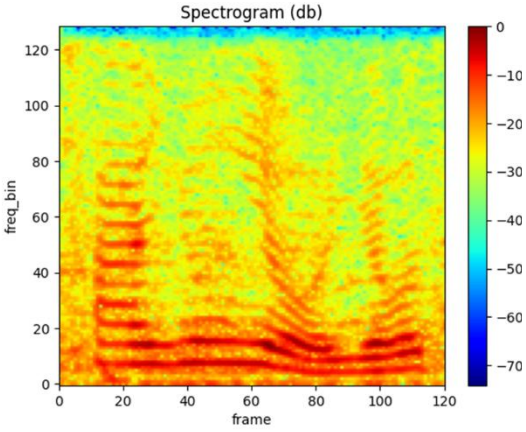
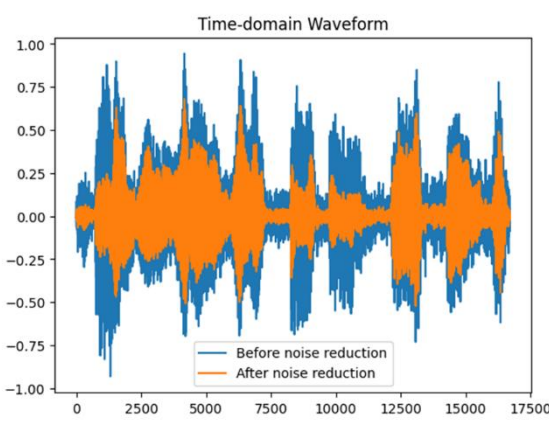
网络总层数	损失函数最终值
1	0.112782
4	0.083670
8	0.083441
12	0.087878
16	0.111613

记录降噪后的时域波形和语谱图如下：



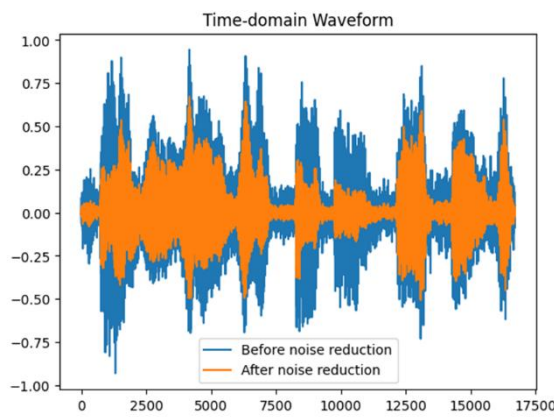
网络总层数=1 时的降噪后时域波形

网络总层数=1 时的降噪后语谱图

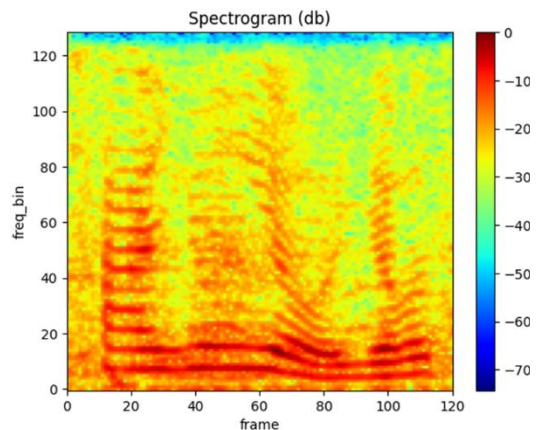


网络总层数=4 时的降噪后时域波形

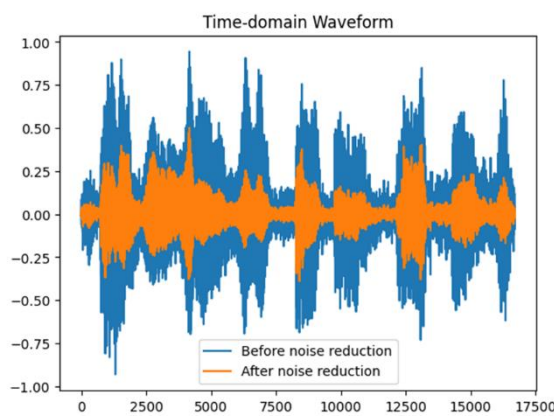
网络总层数=4 时的降噪后语谱图



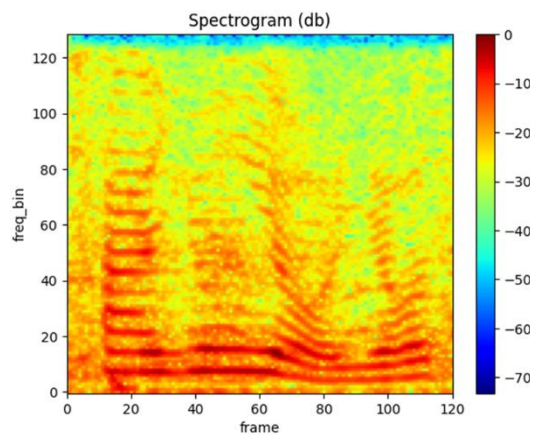
网络总层数=8 时的降噪后时域波形



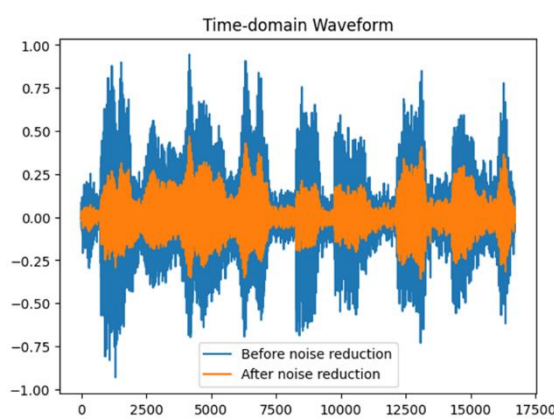
网络总层数=8 时的降噪后语谱图



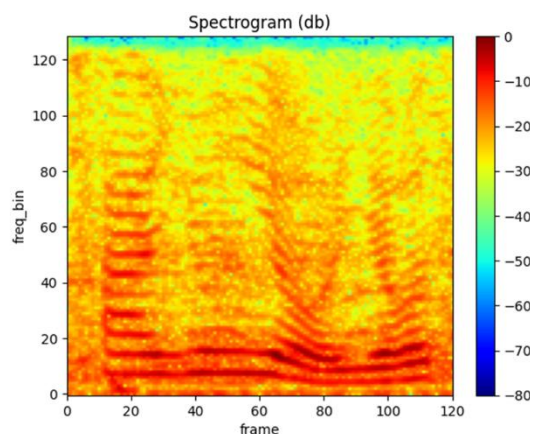
网络总层数=12 时的降噪后时域波形



网络总层数=12 时的降噪后语谱图



网络总层数=16 时的降噪后时域波形



网络总层数=16 时的降噪后语谱图

可以看到当网络层数为1时,训练效果并不太理想。当网络改为堆叠RNN结构时,网络性能有了明显改善。但是当网络层数进一步增加时,性能改善并没有十分明显,反而过深的网络可能导致性能的下降。说明对本实验任务来说,4层左右的网络已经可满足基本要求,没有必要进一步加深网络。

四、实验心得

实验过程中遇到的什么问题。尝试使用什么方法去解决。通过实验获得了什么感悟与理解。

- 1) Torchaudio.load 无法使用，该为零 librosa.load 后就可以正常运行。
- 2) np.ndarray 没有 dim 这个属性，如果需要调用 dim 这个属性需要先将 np.ndattay 对象用 torch.FloatTensor()函数作用改为 torch 对象。
- 3) 若使用 GPU 则需要在虚拟环境 TensorFlow-gpu 中手动安装很多 python 包，而使用 CPU 的基环境的 python 包是齐全的。
- 4) 在绘图时我将降噪前和降噪后的时域波形绘制在同一张图内，可以更好地对比以看出降噪效果。